

System Architecture Report: AUNTY D

Status: Implementation Overview

Architecture Pattern: Client-Server

1. Architectural Overview

The AUNTY D system is designed as a decoupled architecture where two independent React-based frontends communicate with a centralized Node.js/Express API. This separation allows for specialized user experiences for borrowers versus administrative staff while maintaining a "single source of truth" at the database level.

2. Component Hierarchy & Logic

2.1 Client-Side Applications

The frontends are built as Single Page Applications (SPAs) that interact with the backend via HTTPS/REST.

- **Customer Portal:** Manages the end-to-end borrower lifecycle, including marketing landing pages, loan product selection, multi-step application flows, and profile management.
- **Admin Dashboard:** A restricted-access interface for internal staff. It facilitates loan oversight, status transitions (approval/rejection), and operational data management. Access is strictly governed by role-based authentication (role === admin).

2.2 Backend API (The Core)

The backend functions as a unified REST API (/api/) that serves as the gateway to the system's data and logic.

- **Authentication Middleware:** Validates JSON Web Tokens (JWT) for all protected routes.
- **Modular Routing:** Logically separated into domains:
 - auth: Security and onboarding.
 - user: Personal data and profile updates.
 - loan: Application processing and history.
 - upload: File handling for supporting documentation.
 - admin: High-level operational commands.

2.3 Data & Storage Layer

The system employs a multi-faceted storage strategy:

- **PostgreSQL:** Stores structured relational data such as user credentials, loan parameters, guarantor information, and transaction logs.
- **Prisma ORM:** Acts as the abstraction layer, ensuring type safety and efficient querying.

- **Filesystem Storage:** Loan-specific documents and PDFs are stored on the server's local disk under the /uploads directory, mapped as static assets for client-side retrieval.

3. Communication & Data Flow

1. **Request Initiation:** The browser triggers an API call (utilizing environment-specific base URLs).
2. **Security Filtering:** The Express server handles CORS and parses incoming JSON or multipart (file) data.
3. **Identity Verification:** Protected routes extract the userId and role from the Bearer Token in the authorization header.
4. **Database Execution:** Prisma translates business logic into SQL queries against the PostgreSQL instance.
5. **Payload Delivery:** Data is returned as JSON; files are served via static links, allowing the browser to open PDFs directly from the backend origin.

4. Deployment Topology & Security

Infrastructure Configuration

Layer	Environment	Key Configuration
Frontends	Static Hosting (e.g., Vercel)	VITE_API_BASE
API	Node.js Process (e.g., Render)	JWT_SECRET, PORT
Database	Managed PostgreSQL (e.g., Neon)	DATABASE_URL
External	SMTP / Google Maps	SMTP_CREDENTIALS, MAPS_API_KEY

Security Posture

- **Data Integrity:** Passwords are never stored in plain text; Bcrypt hashing is mandatory.
- **Least Privilege:** Administrative functionality is walled off behind both JWT verification and server-side role checks.
- **Environment Isolation:** Sensitive credentials and connection strings are strictly managed via environment variables and are excluded from version control.